



SR UNIVERSITY

Campus Warangal

Program: II - B. Tech (CS& AI) Batch:

Name: A.Tejeswar Reddy

Reg. No: 2503A52464

Department: Computer Science and AI Semester: I

AI Assisted Coding-Assignment Date:

TASK 1 – Improve Readability

1. Original code

```
a = 80
```

```
b = 70
```

```
c = 90
```

```
x = (a + b + c) / 3
```

```
if x >= 50:
```

```
    print("pass")
```

```
else:
```

```
    print("fail")
```

2. Prompt

Rewrite this Python code to improve readability. Use meaningful variable names, create a function called `calculate_average()`, improve formatting, and keep the same functionality. Also show the result so I can verify that both versions produce the same output.

3. AI Suggestions

The code can be improved by:

Replacing a, b, and c with meaningful names such as `subject1`, `subject2`, and `subject3`.

Creating a `calculate_average()` function.

Using clear indentation and spacing.

Storing the result in a descriptive variable such as average_marks.

1. Refactored code

```
def calculate_average(first_mark, second_mark, third_mark):  
    return (first_mark + second_mark + third_mark) / 3  
  
def main():  
    math_mark = 80  
    science_mark = 70  
    english_mark = 90  
  
    average = calculate_average(math_mark, science_mark, english_mark)  
  
    if average >= 50:  
        print("pass")  
    else:  
        print("fail")  
  
if __name__ == "__main__":  
    main()
```

2. Test Output

Pass

6.Explanation

The refactored code is easier to understand b

a, b, c → mark1, mark2, mark3

x → average

The average calculation is placed inside calculate_average().

The formatting is clearer.

The original and refactored programs produce the same result, so the original behavior is preserved.

TASK 2 – Remove Repeated Code

1.Original code

name1 = "Arun"

marks1 = 85

if marks1 >= 50:

```
print(name1, "Pass")
else:
    print(name1, "Fail")
```

```
name2 = "Priya"
marks2 = 72
```

```
if marks2 >= 50:
    print(name2, "Pass")
else:
    print(name2, "Fail")
```

2.Prompt

Identify the duplicated logic in this Python code and refactor it into a reusable function. Test the function with at least four students. Make sure the output remains correct and explain why removing duplication improves maintainability.

3.AI Syggestion

The same if-else logic is repeated for every student
This repeated logic should be moved into a reusable function such as `check_result(name, marks)`.

Then the function can be called for any number of students.

4.Refactored code

```
def check_pass_or_fail(student_name, marks):
    if marks >= 50:
        result = "Pass"
    else:
        result = "Fail"

    print(student_name, result)
    return result
```

```
students = [  
    ("Arun", 85),  
    ("Priya", 72),  
    ("Karan", 45),  
    ("Meera", 50),  
    ("Ravi", 39),  
]
```

```
for student_name, marks in students:  
    check_pass_or_fail(student_name, marks)  
else:  
    print(name2, "Fail")
```

5. Test Output

Arun Pass

Priya Pass

Rahul Fail

Sneha Pass

6. Explanation

The original code contains **duplicated logic** because the same if-else statement is written separately for each student.

The refactored version uses one reusable function, `check_result()`.

Advantages of removing duplication:

Less code: We write the logic only once.

Easy maintenance: If the pass mark changes from 50 to 40, we only change it in one place.

Better readability: The purpose of the code is easier to understand.

Reusability: The same function can be used for any number of students.

Fewer errors: There is less chance of making inconsistent changes in repeated code.

TASK 3 – Refactor a Long Conditional

1.Original code

```
age = 68
```

```
temperature = 39
```

```
if age >= 60 and temperature >= 38:
```

```
    print("High Priority")
```

```
elif age >= 60 and temperature < 38:
```

```
    print("Medium Priority")
```

```
elif age < 60 and temperature >= 38:
```

```
    print("Medium Priority")
```

```
else:
```

```
    print("Normal Priority")
```

2.Prompt

Simplify this Python conditional logic without changing its behavior. Create a function called `determine_priority(age, temperature)`. Test it with at least four different age and temperature combinations. Compare the original and refactored logic and explain whether the behavior is preserved.

3.AI Suggestion

The original code has four conditions, but the last three can be simplified.

The logic is:

High Priority → age is 60 or above **and** temperature is 38 or above.

Medium Priority → only one of these conditions is true.

Normal Priority → neither condition is true.

Therefore, we can simplify the conditional by checking the two important conditions directly.

4.Refactored code

```
def determine_priority_original(age, temperature):
```

```
    if age >= 60 and temperature >= 38:
```

```
    return "High Priority"
elif age >= 60 and temperature < 38:
    return "Medium Priority"
elif age < 60 and temperature >= 38:
    return "Medium Priority"
else:
    return "Normal Priority"
```

```
def determine_priority(age, temperature):
    if age >= 60 and temperature >= 38:
        return "High Priority"
    elif age >= 60 or temperature >= 38:
        return "Medium Priority"
    else:
        return "Normal Priority"
```

```
test_cases = [
    (68, 39), # High Priority
    (68, 37), # Medium Priority
    (50, 39), # Medium Priority
    (50, 37), # Normal Priority
    (60, 38), # High Priority
    (59, 38), # Medium Priority
]
```

```
for age, temperature in test_cases:
```

```
original_result = determine_priority_original(age, temperature)
refactored_result = determine_priority(age, temperature)

print(f"Age: {age}, Temperature: {temperature} -> Original: {original_result},
Refactored: {refactored_result}, Match: {original_result == refactored_result}")
```

5. Test Output

High Priority

Medium Priority

Medium Priority

Normal Priority

6. Explanation

The original code checks `age >= 60` and `temperature >= 38` repeatedly. This makes the code longer and harder to maintain.

The refactored code:

- Uses a reusable `determine_priority()` function.

- Gives meaningful names to conditions.

- Removes repeated comparisons.

- Makes the logic easier to understand.

- Produces the same results as the original code.

TASK 4 – Refactor Inefficient List Processing

1. Original code

```
patient_ids = [101, 205, 309, 412, 518, 623]
```

```
target = 412
```

```
found = False
```

```
for patient_id in patient_ids:
```

```
    if patient_id == target:
```

```
        found = True
```

```
if found:
```

```
print("Patient Found")
else:
    print("Patient Not Found")
```

2.Prompt

Improve the readability of this Python code. Also explain whether a list, set, or dictionary is better if I need to perform frequent patient ID lookups. Compare their typical lookup time complexities and recommend the most appropriate data structure. Then provide refactored code and test it with both an existing and a missing patient ID.

3.AI Suggestion

The loop can be replaced with Python's in operator, which is much easier to read.

For **frequent lookups**, a **set** is the best choice when we only need to know whether a patient ID exists:

List: $O(n)$ average/worst-case lookup because elements may need to be checked one by one.

Set: $O(1)$ average lookup, with $O(n)$ worst case in unusual hash-collision situations.

Dictionary: $O(1)$ average key lookup, but it is more appropriate when each patient ID needs to map to additional information.

Since this program only checks whether an ID exists, a **set** is justified.

4.Refactored code

```
patient_ids = {101, 205, 309, 412, 518, 623}
```

```
def check_patient(patient_id):
```

```
    if patient_id in patient_ids:
```

```
        print("Patient Found")
```

```
    else:
```

```
        print("Patient Not Found")
```

```
check_patient(412)
```

```
check_patient(999)
```

5. Test Output
Patient Found
Patient Not Found

6. Explanation

The original code manually loops through every patient ID and uses a Boolean variable to track whether the target was found. This is unnecessarily verbose.

TASK 5 – Complete Legacy Code Review with AI

1. Original code

```
students = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]
```

```
total = 0
```

```
for s in students:
```

```
    total = total + s[1]
```

```
avg = total / len(students)
```

```
for s in students:
```

```
    if s[1] >= 50:
```

```
        print(s[0], "Pass")
```

```
    else:
```

```
        print(s[0], "Fail")
```

```
print("Average =", avg)
```

2. Prompt

Review this legacy Python code and identify at least five improvements. Refactor it using meaningful variable names and reusable functions. Use suitable Python features such as `sum()`, `len()`, comprehensions, or f-strings where they improve clarity. Make sure the refactored version produces the same results as the original. Also explain which suggested improvements should be accepted or rejected and why.

3. AI suggestion

The code should be refactored mainly for readability, reusability, and maintainability.

The most important improvements are meaningful variable names, reusable functions, `sum()`, tuple unpacking, and a named constant for the pass mark.

4. Refactored code

```
PASSING_SCORE = 50
```

```
student_records = [("Arun", 78), ("Priya", 45), ("Rahul", 88), ("Meena", 52)]
```

```
def calculate_average(records):
    total_score = sum(score for _, score in records)
    return total_score / len(records)

def print_student_results(records):
    for student_name, score in records:
        result = "Pass" if score >= PASSING_SCORE else "Fail"
        print(f"{student_name} {result}")

average_score = calculate_average(records)
print(f"Average = {average_score}")

print_student_results(student_records)
```

5. Test Output

Arun Pass

Priya Fail

Rahul Pass

Meena Pass

Average = 65.75

6. Explanation

The refactored code is easier to maintain because each function has a clear responsibility:

`calculate_average()` calculates the average marks.

`get_result()` determines whether a student passes or fails.

`display_results()` displays each student's result.

PASS_MARK gives the value 50 a meaningful name.

sum() replaces the manual total-accumulation loop.

f-strings make the output easier to read.

Meaningful names such as **student_name**, **mark**, and **total_marks** replace vague names such as **s** and **total**.